

The background features two abstract network graphs. One graph in the top right corner consists of green nodes and connecting lines. Another graph in the bottom left corner consists of blue nodes and connecting lines. Both graphs are composed of interconnected circles of varying sizes, representing a complex network structure.

WQFS

(World event & Quest Fusion System)

월드 이벤트 & 퀘스트
융합 시스템
(10주차)

202444086 이인희

지난 주 진행 상황

- WQFS 퀘스트 목록

- UI로 퀘스트를 표시할 수 있게 퀘스트 리스트를 문자열 형태로 반환

```
std::vector<std::string> WQFS::GetQuestListByString(int maxQuestList) {
    std::vector<std::string> stringQuest;

    for (const auto quest : questList) {
        if (stringQuest.size() >= maxQuestList) {
            break;
        }

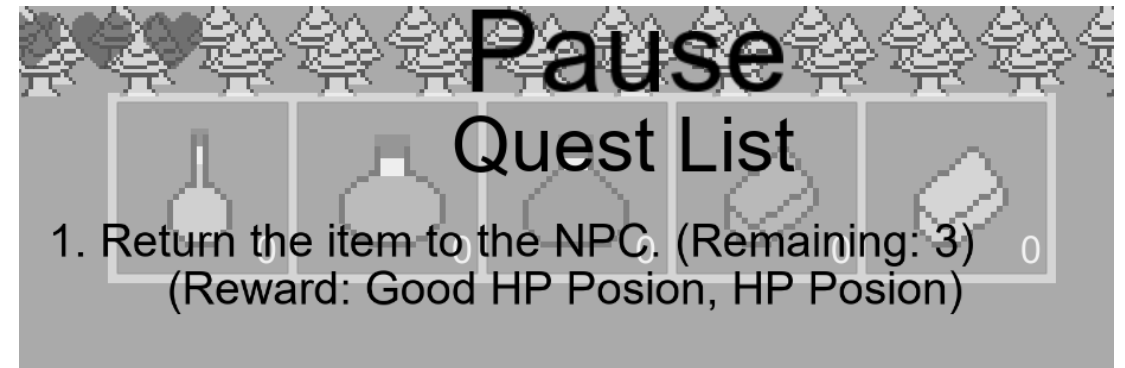
        std::string strData = "";
        for (const auto target : QuestTargetObjects) {
            if (target.first->getQuestNumber() != -1 && quest.first == target.first->getQuestNumber()) {
                strData = std::to_string(quest.first + 1) + " ";
                switch (std::get<1>(quest.second)) {
                    case 0: strData += "Save NPC.";
                        break;
                    case 1: strData += "Retrieve NPC's Item.";
                        break;
                    case 2: strData += "Return the item to the NPC. (Remaining: " + std::to_string(WQFS::GetInstance().questRemaining[quest.first]) + ")";
                        break;
                    case 3: strData += "Remove obstacles for NPC (Remaining: " + std::to_string(WQFS::GetInstance().questRemaining[quest.first]) + ")";
                        break;
                    default: strData += " "; break;
                }
            }
            break;
        }

        if (strData != "") {
            stringQuest.push_back(strData);

            strData = "(Reward: " + std::get<0>(quest.second)[0].GetName();
            for (int i = 1; i < std::get<0>(quest.second).size(); ++i) {
                strData += ", " + std::get<0>(quest.second)[i].GetName();
            }
            strData += ")";

            stringQuest.push_back(strData);
        }
    }

    if (stringQuest.size() > 0) {
        return stringQuest;
    }
    else {
        stringQuest.push_back("");
        return stringQuest;
    }
}
```



0%

90%

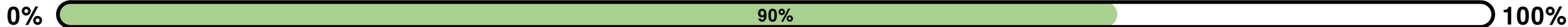
100%

지난 주 진행 상황

- WQFS 퀘스트 타입 1

- 퀘스트 타입에 따라 발생시키는 이벤트와 완료 조건이 다름

타입	내용	발생 이벤트	완료 조건
0	NPC 구출	몬스터	위협 몬스터 제거
1	도둑 맞은 아이템 되찾기	몬스터	훔친 몬스터 제거
2	아이템 줍기	동/정적 이벤트	NPC 주변에 떨어진 아이템 줍기
3	장애물 제거	동/정적 이벤트	NPC 주변에 있는 장애물 제거



지난 주 진행 상황

• WQFS 퀘스트 타입 2

```
void WQFS::MakeQuest(NPC &npc, WorldEvent &event) {
    int questType = 0;

    int random = rand() % 2;

    if (event.GetType() == 0) {
        if (random == 0) questType = 0;
        else questType = 1;
    }
    else {
        if (random == 0) questType = 2;
        else questType = 3;
    }

    WQFS::GetInstance().SetCompensation(npc, event, questType);

    if (questType == 2) {
        questRemaining[npc.getQuestNumber()] = (rand() % 3) + 3;
    }
    if (questType == 3) {
        questRemaining[npc.getQuestNumber()] = (rand() % 3) + 1;
    }

    std::cout << "보상 ";

    for (int i = 0; i < std::get<0>(WQFS::GetInstance().questList[WQFS::GetInstance().questNumber - 1]).size(); i++) {
        std::cout << std::get<0>(WQFS::GetInstance().questList[WQFS::GetInstance().questNumber - 1])[i].GetName() << " ";
    }

    std::cout << npc.GetType() << " " << event.GetType() << std::endl;
}
```



0%

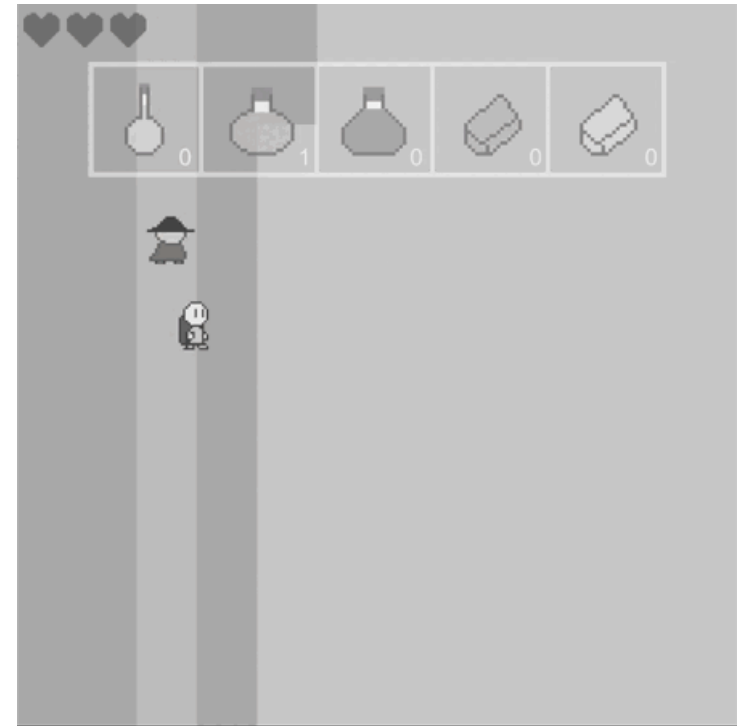
90%

100%

지난 주 진행 상황

- WQFS 쓰나미 이벤트
 - 평소에는 정지해 있다가 이벤트 발생시 이동

```
for (auto& def : defaultPosition) {  
    if (WQFS::GetInstance().GetEvent(def.first).GetIsCanCollid()) {  
        switch (std::get<1>(def.second)) {  
            case 0: eventObjects[def.first].objPosition.x -= 300 * dt * timeScale; break;  
            case 1: eventObjects[def.first].objPosition.x += 300 * dt * timeScale; break;  
            case 2: eventObjects[def.first].objPosition.y -= 300 * dt * timeScale; break;  
            case 3: eventObjects[def.first].objPosition.y += 300 * dt * timeScale; break;  
        }  
  
        WQFS::GetInstance().GetEvent(def.first).SetPosition(eventObjects[def.first].objPosition.x, eventObjects[def.first].objPosition.y);  
    }  
    else {  
        eventObjects[def.first].objPosition = std::get<0>(def.second);  
        WQFS::GetInstance().GetEvent(def.first).SetPosition(eventObjects[def.first].objPosition.x, eventObjects[def.first].objPosition.y);  
    }  
}
```



0%

90%

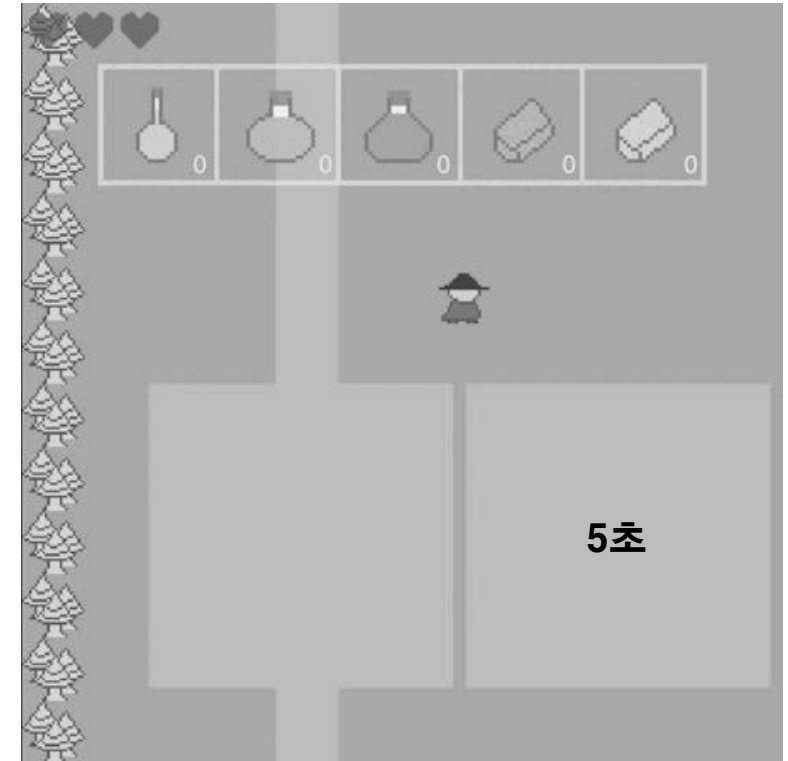
100%

지난 주 진행 상황

- WQFS 이벤트 발생 동적화
 - 고정적으로 발생하는 이벤트를 동적화하여 예측하기 힘들게 함

```
errorTime = ((rand() % int(maxTime)) + 1) + errorValue;  
if (errorTime < errorValue) {  
    errorTime = errorValue;  
}
```

```
if (!doEvent && eventTimer > errorTime + pauseDuration) {  
    //std::cout << "이벤트 시작" << std::endl;  
    eventStart = clock();  
    doEvent = true;  
}
```



0%  90%  100%

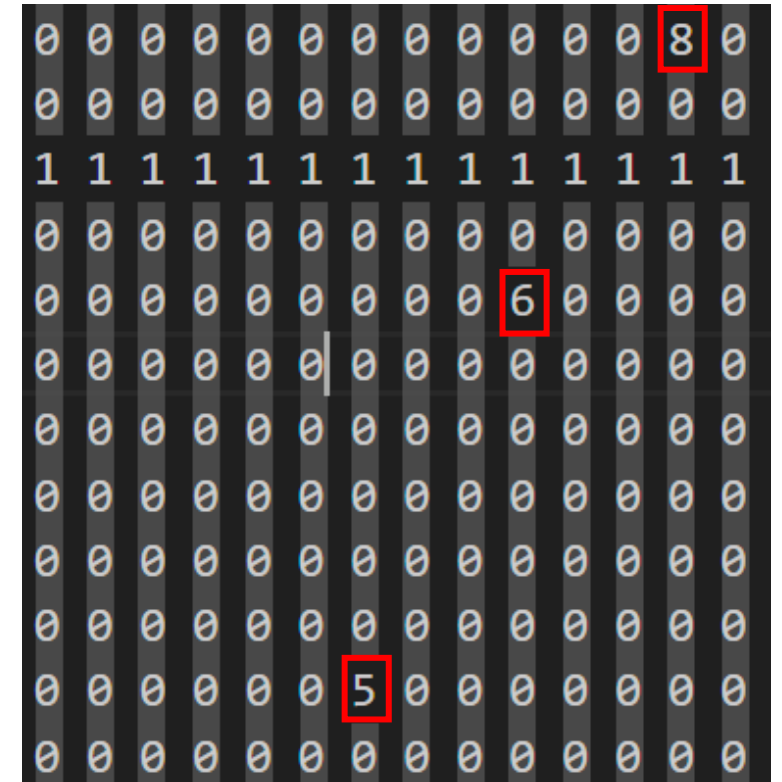
진행 상황

- WQFS 서브 타입

- 개발자가 정의 타입, 필요 시 개발자가 원하는 형태로 사용 가능

- 예) LevelGenerator에서 이벤트 위치 동기화

```
WQFS::GetInstance().AddNPC("Normal", 0, 0, 0.0f, 0.0f, ...  
WQFS::GetInstance().AddEvent("Monster", 0, 0, 2, 0.0f, 0.0f, ...  
WQFS::GetInstance().AddEvent("Landslide", 1, 0, 2, 0.0f, 0.0f, ...  
WQFS::GetInstance().AddEvent("Earthquake", 1, 1, 2, 0.0f, 0.0f, ...  
WQFS::GetInstance().AddEvent("Tsunami", 2, 2, 2, 0.0f, 0.0f, ...  
  
case 5: //Landslide event Object  
    WQFS::GetInstance().GetEvent(events[0]).SetPosition(pos.x, pos.y);  
    obj = GameObject(ResourceManager::GetTexture("MountainTile"), pos, size, 0.0f, glm::vec3(1.0f));  
    walls.push_back(obj);  
    break;  
case 6: //Earthquake event Object  
    WQFS::GetInstance().GetEvent(events[1]).SetPosition(pos.x, pos.y);  
    break;  
case 7: //Tsunami Object  
    WQFS::GetInstance().GetEvent(events[2]).SetPosition(pos.x, pos.y);  
    obj = GameObject(ResourceManager::GetTexture("GroundTile"), pos, size, 0.0f, glm::vec3(200 / 255.0f));  
    walls.push_back(obj);  
    break;  
case 8: //Monster Object  
    WQFS::GetInstance().GetEvent(monsters[0]).SetPosition(pos.x, pos.y);  
    break;  
case 9: //NPC Object  
    WQFS::GetInstance().GetNPC(npcs[0]).SetPosition(pos.x, pos.y);  
    break;
```



0%

90%

100%

진행 상황

- 태크 데모 인벤토리
 - 아이템 리스트와 현재 소지 개수 표기



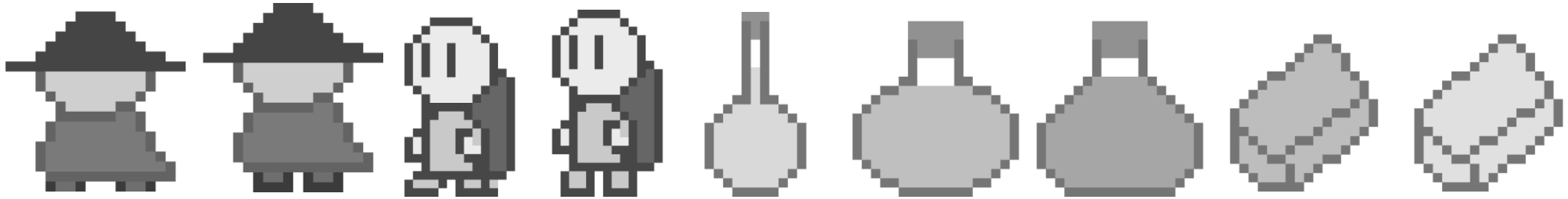
```
InventoryObject->objPosition.x = 100.0f + cameraPos.x;
InventoryObject->objPosition.y = 80.0f + cameraPos.y;
InventoryObject->Draw(*sRenderer);

for (auto item : inventory) {
    std::stringstream itemCnt;
    itemCnt << item.second;
    itemObjects[item.first.GetName()].objPosition.x = inventoryItemPos[cnt].x + cameraPos.x;
    itemObjects[item.first.GetName()].objPosition.y = inventoryItemPos[cnt].y + cameraPos.y;
    itemObjects[item.first.GetName()].Draw(*sRenderer);
    textRenderer->renderText(itemCnt.str(), inventoryItemPos[cnt].x + 100.0f, inventoryItemPos[cnt].y + 90.0f, 0.5f, glm::vec3(0.95f));
    ++cnt;
}
```

0%  90%  100%

진행 상황

- 태크 데모 스프라이트



플레이어

NPC 1

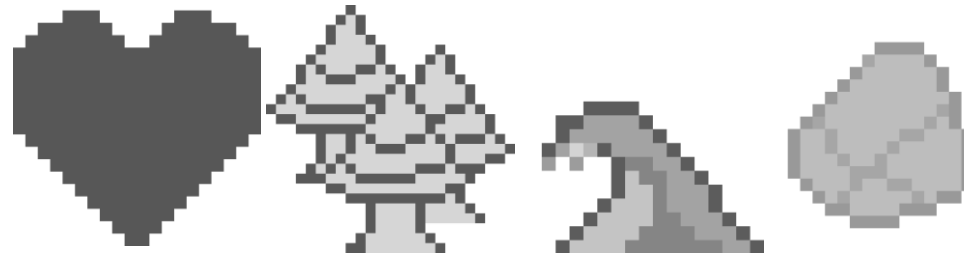
HP 포션

좋은 HP 포션

마나 업
포션

은

금



HP

나무

쓰나미

낙석

0%

90%

100%

추후 일정

- 현재 구현된 기본 정적, 동적 이벤트를 기반으로 태풍 등 다양한 이벤트를 구현할 수 있게 기능 추가 및 테스트
- NPC, 기타 UI 요소나 오브젝트 등의 스프라이트 제작
- 배경음악, 사운드 이펙트 적용